

Cyprus University of Technology

CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Δομές διακλάδωσης, Δομές Επανάληψης, Πίνακες

Παναγιώτης Ηλία

Λογικοί τελεστές

- Λογικοί τελεστές για λογικές εκφράσεις
  - Άρνηση: **`!B`**
  - ΚΑΙ: **`(A && B)`**
  - Ή: **`(A || B)`**
- Έλεγχος για βασικούς τύπους Α,Β:
  - Ισότητας: **`(A == B)`**
  - Ανισότητας: **`(A != B)`** ή **`(!(A == B))`**
  - Μεγαλύτερο/Μικρότερο ή ίσο: **`(A <= B)`**, **`(A >= B)`**
- Έλεγχος για μεταβλητές (αντικείμενα) οποιουδήποτε άλλου τύπου γίνεται με την μέθοδο equals (πρέπει να έχει οριστεί):
  - Ισότητας: **`(A.equals(B))`**
  - Ανισότητας: **`(!A.equals(B))`**

Λογικές σταθερές:

- **true: αληθές**
- **false: ψευδές**

Έλεγχος ισότητας για Strings

- Για να ελέγξουμε αν δύο μεταβλητές String έχουν την ίδια τιμή πρέπει να χρησιμοποιήσουμε την μέθοδο equals
  - Παράδειγμα:

```
String firstString = "abc";
String secondString = "ABC";
boolean test1 = firstString.equals(secondString);
boolean test2 = firstString.equals("abc");
```

  - Η παρακάτω εντολή δεν είναι σωστή

```
boolean test3 = (firstString == secondString);
```

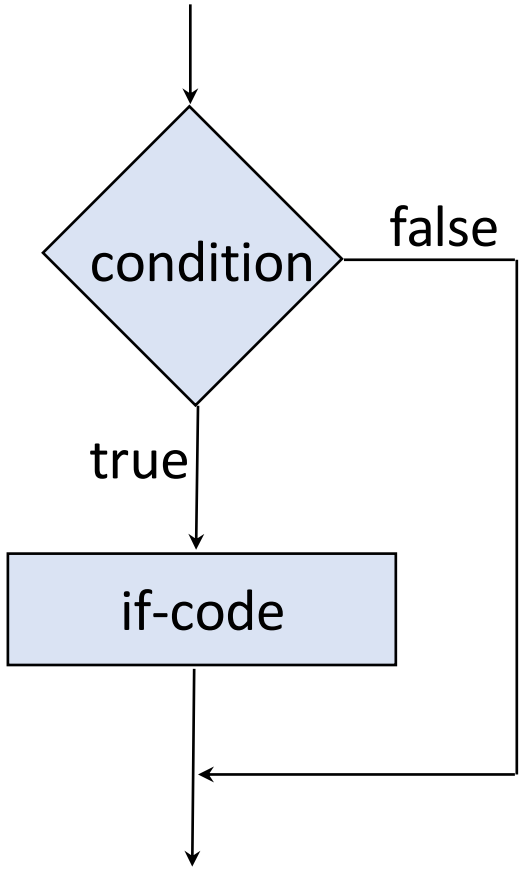
  - Περνάει από τον compiler και σε κάποιες περιπτώσεις θα δουλέψει αλλά δεν κάνει αυτό που θέλουμε

Βρόγχοι – If-then Statement

- Στην Java το if-then statement έχει το εξής συντακτικό

```
if (condition)
{
    …if-code block…
}
```

  - Αν η συνθήκη είναι αληθής (true) τότε εκτελείται το block κώδικα if-code
  - Αν η συνθήκη είναι ψευδής (false) τότε το κομμάτι αυτό προσπερνιέται και συνεχίζεται η εκτέλεση.



```
import java.util.Scanner;

class IfTest1
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt = reader.nextInt();
        if (inputInt > 0){
            System.out.println(inputInt + " is positive");
        }
    }
}
```

```
import java.util.Scanner;

class IfTest1b
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt = reader.nextInt();
        boolean inputIsPositive = (inputInt > 0)
        if (inputIsPositive == true){
            System.out.println(inputInt + " is positive");
        }
    }
}
```

Ακόμη και αν δεν το προσδιορίσουμε ελέγχει ισότητα

```
import java.util.Scanner;

class IfTest1b
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt = reader.nextInt();
        boolean inputIsPositive = (inputInt > 0)
        if (inputIsPositive){
            System.out.println(inputInt + " is positive");
        }
    }
}
```

Programming Style: Λογικές μεταβλητές

- Συνηθίζεται όταν ορίζουμε λογικές μεταβλητές το όνομα τους να είναι αυτό που εκφράζει την περίπτωση που η μεταβλητή αποτιμάται true

```
int X = 10;
boolean isPositive = (X > 0);
boolean isNegative = (X < 0);
boolean isNotPositive = !isPositive;
```

- Αυτό βολεύει για την εύκολη ανάγνωση του προγράμματος όταν χρησιμοποιούμε την μεταβλητή

```
if(isPositive){
   System.out.println("Variable is positive");
}
```

- Το ίδιο ισχύει και όταν αργότερα θα ορίζουμε μεθόδους που επιστρέφουν λογικές τιμές
  - Π.χ., για τα Strings υπάρχει η μέθοδος equals που γίνεται true όταν έχουμε ισότητα και η μέθοδος isEmpty που είναι true όταν έχουμε άδειο String

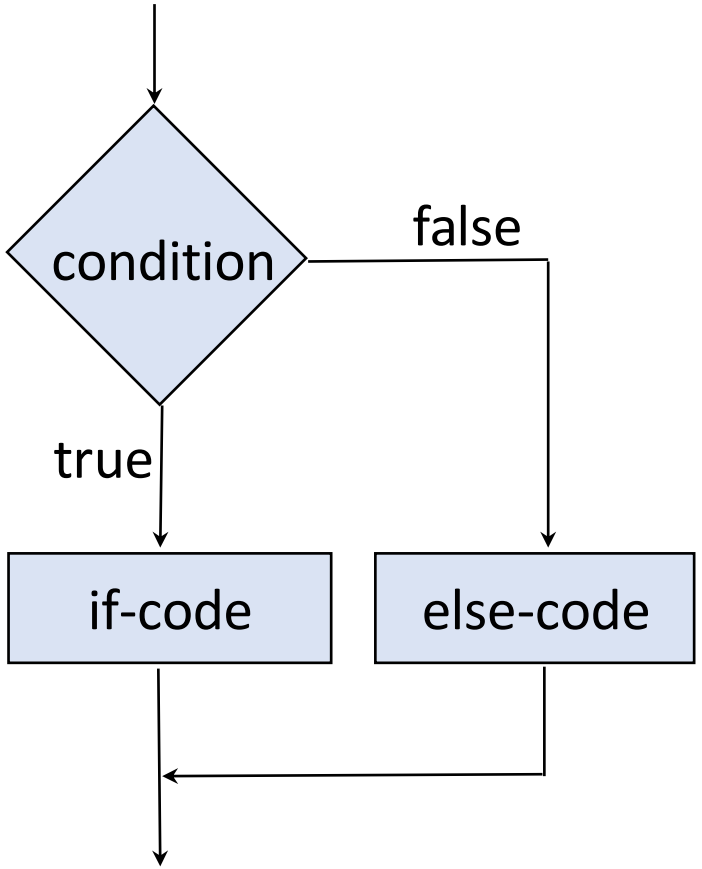
Βρόγχοι – if-then-else Statement

- Στην Java το if-then-else statement έχει το εξής συντακτικό

```
if (condition){
    …if-code block…
}else{
    …else-code block…
}
```

  - Αν η συνθήκη είναι αληθής εκτελείται το block **if-code**
  - Αν η συνθήκη είναι ψευδής εκτελείται το block **else-code**

- Ο κώδικας του **if-code block** ή του **else-code block** μπορεί να περιέχουν άλλο (φωλιασμένο (nested)) if statement
  - Προσοχή: ένα **`else`** ταιριάζεται με το τελευταίο ελεύθερο **`if`** ακόμη κι αν η στοίχιση του κώδικα υπονοεί διαφορετικά



```
import java.util.Scanner;

class IfTest2
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt = reader.nextInt();
         if (inputInt > 0){
               System.out.println(inputInt + " is positive");
         }else{
               System.out.println(inputInt + " is not positive");
         }
    }
}
```

```
import java.util.Scanner;

class IfTest3
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt = reader.nextInt();
         if (inputInt > 0){
               System.out.println(inputInt + " is positive");
         }else if (inputInt < 0){
                System.out.println(inputInt + " is not positive");
         }else{
                System.out.println(inputInt + " is zero");
         }
    }
}
```

Προσοχή!

ΛΑΘΟΣ!

```
if( i == j )
    if ( j == k )
        System.out.print("i equals k");
else
    System.out.print("i is not equal to j");
```

Το **else** μοιάζει σαν να πηγαίνει με το **μπλε if** αλλά ταιριάζεται με το τελευταίο (**πράσινο**) **if**

Πάντα να βάζετε { } στο σώμα των if-then-else statements
Πάντα να στοιχίζετε σωστά τον κώδικα.

Προσοχή!

ΣΩΣΤΟ!

```
if( i == j ){
    if ( j == k ){
        System.out.print("i equals k");
    }
}
else {
    System.out.print("i is not equal to j");
}
```

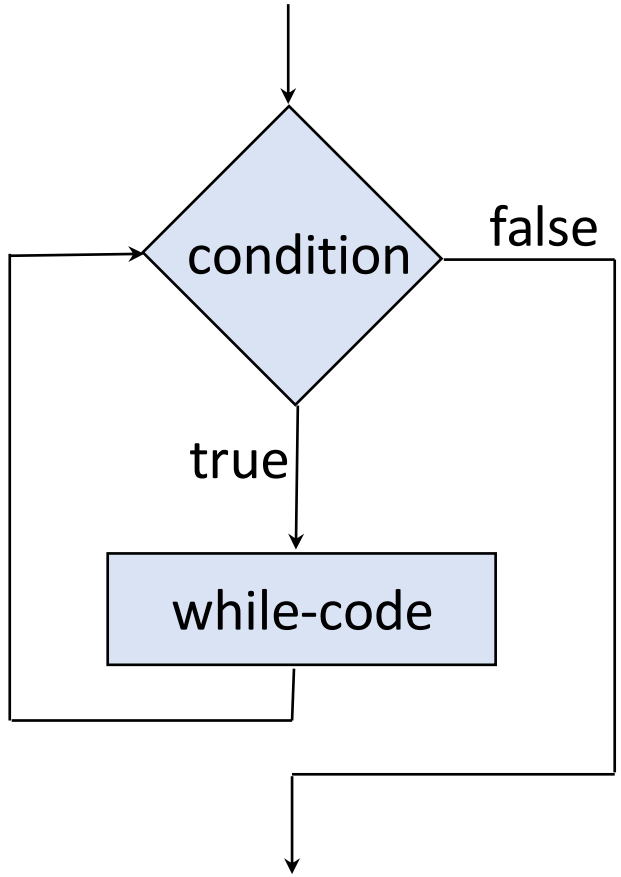
Το **else** μοιάζει σαν να πηγαίνει με το **μπλε if** αλλά ταιριάζεται με το τελευταίο (**πράσινο**) **if**

Πάντα να βάζετε { } στο σώμα των if-then-else statements
Πάντα να στοιχίζετε σωστά τον κώδικα.

Επαναλήψεις - While statement

- Στην Java το while statement έχει το εξής συντακτικό

```
while(condition)
{
    …while-code block…
}
```



- Αν η συνθήκη είναι αληθής τότε εκτελείται το block κώδικα while-code
- Ο while-code block κώδικας υλοποιεί τις επαναλήψεις και αλλάζει την συνθήκη
- Στο τέλος του while-code block η συνθήκη αξιολογείται εκ νέου

Παράδειγμα

```
Scanner inputReader = new Scanner(System.in);
String input = inputReader.next();

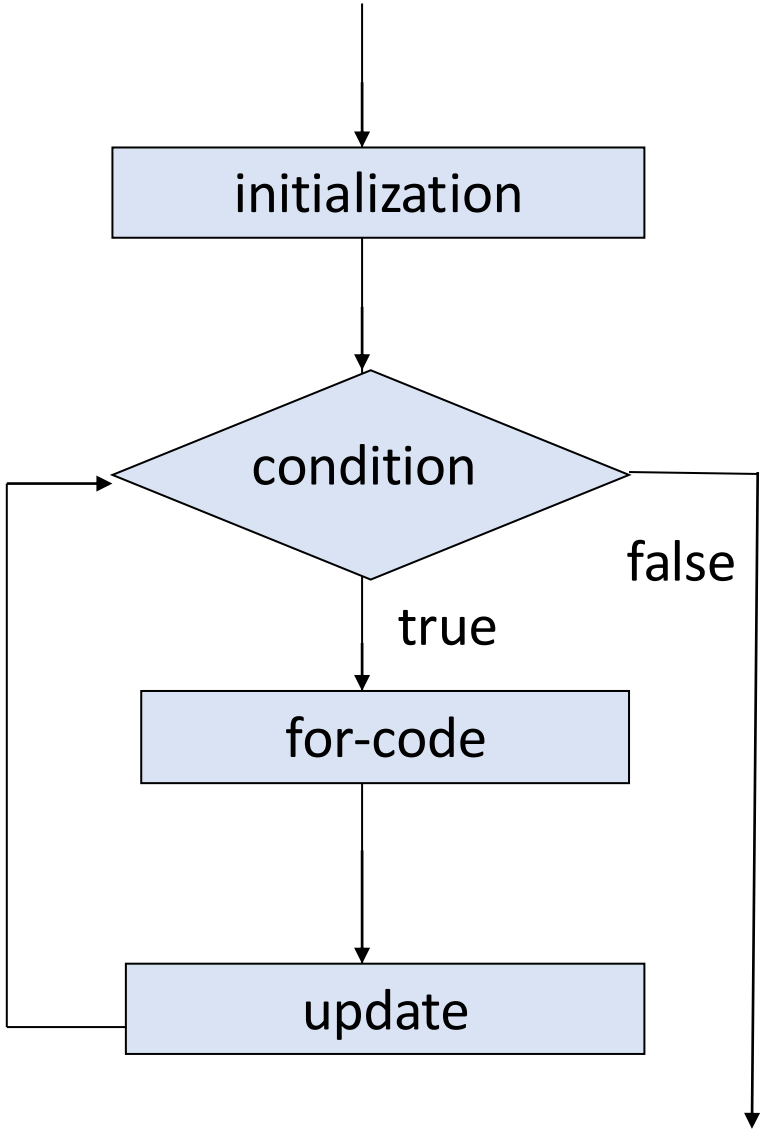
while(input.equals("Yes"))
{
    System.out.println("Do you want to continue?");
    input = inputReader.next();
}
```

Επαναλήψεις – for statement

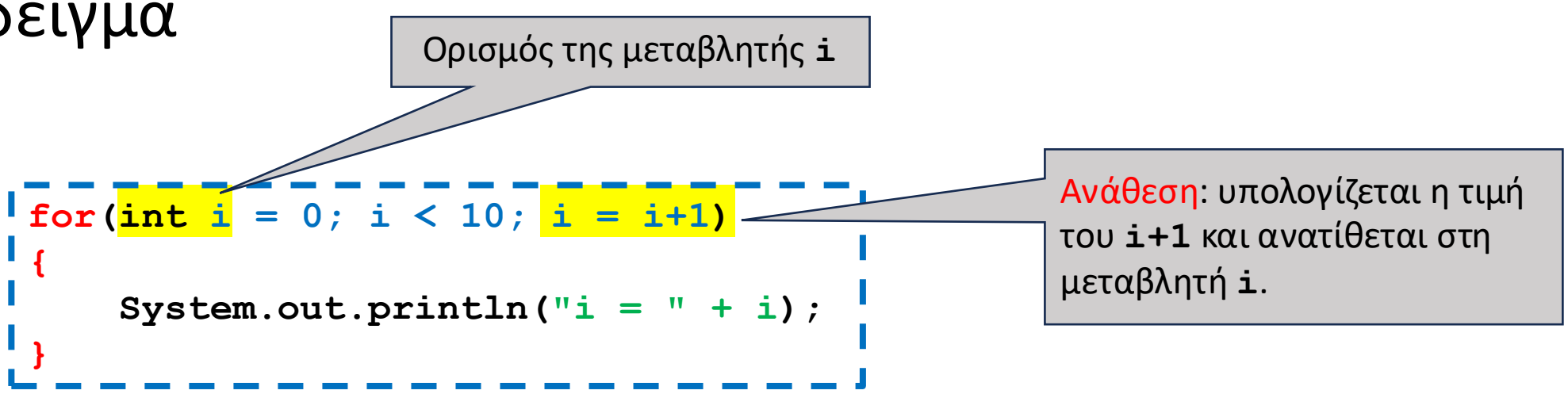
- Στην Java το for statement έχει το εξής συντακτικό

```
for(intialization; condition; update)
{
    …for-code block…
}
```

- Το όρισμα του for έχει 3 κομμάτια χωρισμένα με ;
  - Την αρχικοποίηση (initialization): εκτελείται μία μόνο φορά
  - Τη λογική συνθήκη (condition): ελέγχεται πριν από κάθε επανάληψη
  - Την ενημέρωση (update): υπολογίζεται μετά το κυρίως σώμα της επανάληψης
  - Ο κώδικας επαναλαμβάνεται μέχρι η συνθήκη να γίνει ψευδής



Παράδειγμα



- Ισοδύναμο με while

```
int i = 0;
while(i < 10)
{
    System.out.println("i = " + i);
    i = i+1;
}
```

Παράδειγμα

```
for(int i = 0; i < 10; i++)
{
    System.out.println("i = " + i);
}
```

`i++` ισοδύναμο με το `i = i+1`

- Ισοδύναμο με while

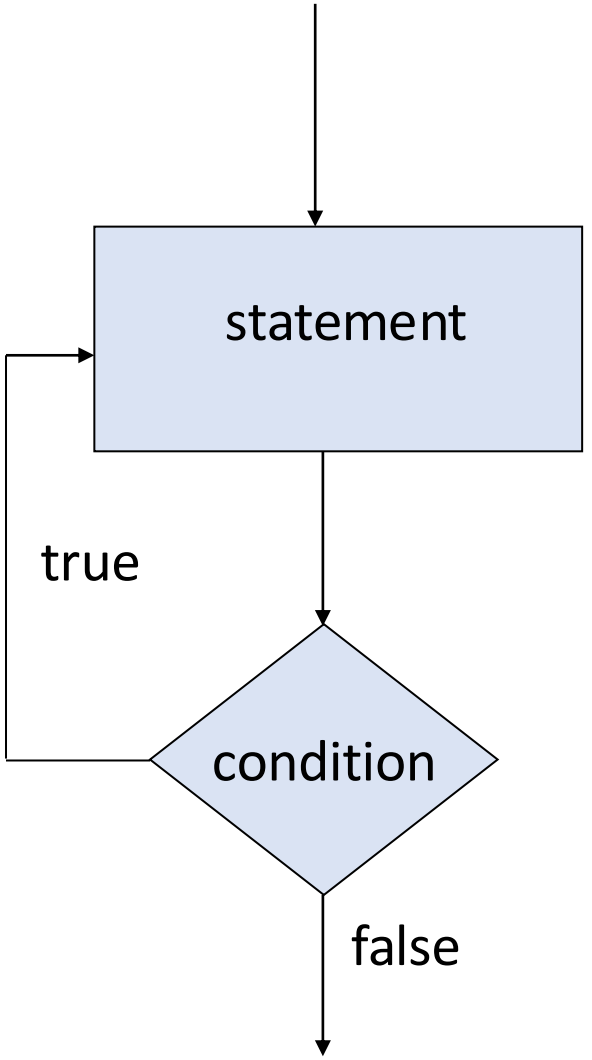
```
int i = 0;
while(i < 10)
{
    System.out.println("i = " + i);
    i++;
}
```

To Do-While statement

- Ένα `do while` statement έχει το εξής συντακτικό:

```
...
do
{
      …while-code block…

}while(condition)
```



- Το while code εκτελείται τουλάχιστον μία φορά
  - Αν η συνθήκη είναι αληθής ο κώδικας εκτελείται ξανά

Παράδειγμα

- Κάνετε πρόγραμμα **που παίρνει σαν είσοδο ένα αριθμό και υλοποιεί μια αντίστροφη μέτρηση**. Αν ο αριθμός είναι θετικός η αντίστροφη μέτρηση γίνεται προς τα κάτω μέχρι το μηδέν, αν είναι αρνητικός γίνεται προς τα πάνω μέχρι το μηδέν. Η διαδικασία επαναλαμβάνεται μέχρι ο χρήστης να δώσει την τιμή μηδέν.

```
import java.util.Scanner;

class FlowTest
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt = reader.nextInt();
        while (inputInt != 0)
        {
            if (inputInt < 0){
                for (int i = inputInt; i < 0; i++)
                 {
                    System.out.println("Counter = " + i);
                 }
             }else if (inputInt > 0){
                for (int i = inputInt; i > 0; i--)
                {
                     System.out.println("Counter = " + i);
                }
             }
             inputInt = reader.nextInt();
         }
    }
}
```

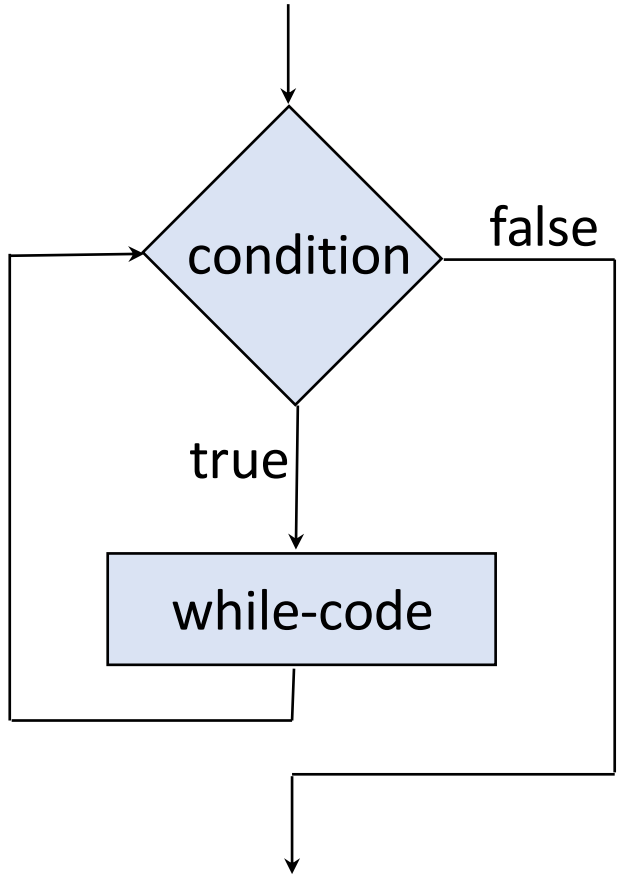
```
import java.util.Scanner;

class FlowTest2
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt;
        do
        {
            inputInt = reader.nextInt();
            if (inputInt < 0 ){
                for (int i = inputInt; i < 0; i++)
                {
                    System.out.println("Counter = " + i);
                }
            } else if (inputInt > 0){
                for (int i = inputInt; i > 0; i--)
                {
                    System.out.println("Counter = " + i);
                }
            }
        }while (inputInt != 0)
    }
}
```

Επαναλήψεις - While statement

- Στην Java το while statement έχει το εξής συντακτικό

```
while(condition)
{
    …while-code block…
}
```



- Αν η συνθήκη είναι αληθής τότε εκτελείται το block κώδικα while-code
- Ο while-code block κώδικας υλοποιεί τις επαναλήψεις και αλλάζει την συνθήκη
- Στο τέλος του while-code block η συνθήκη αξιολογείται εκ νέου
- **Ο κώδικας επαναλαμβάνεται μέχρι η συνθήκη να γίνει ψευδής**

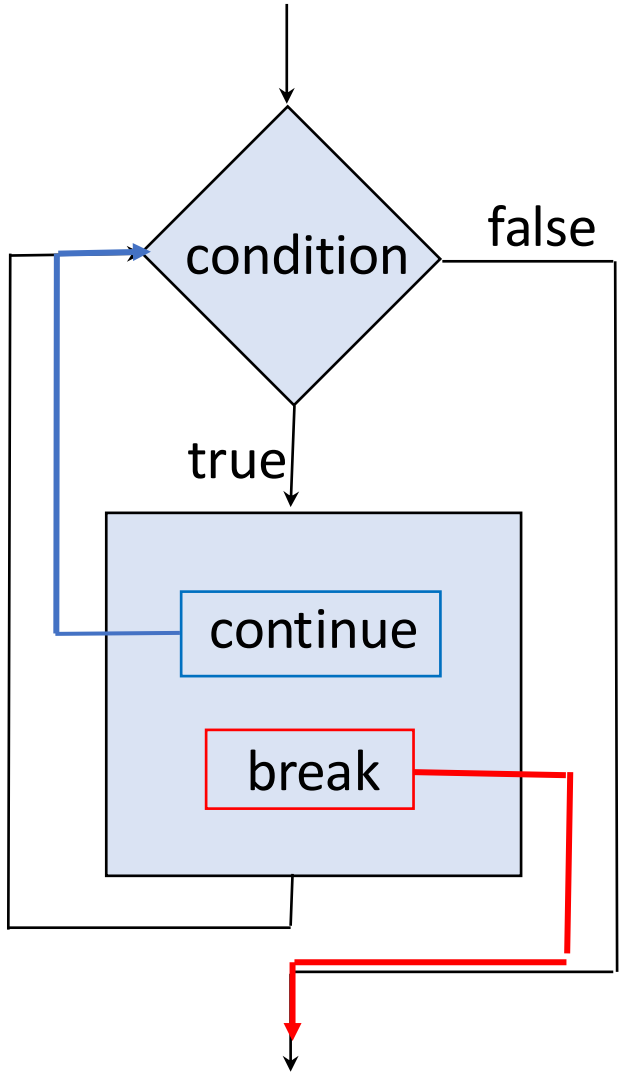
Οι εντολές break και continue

- continue: Επιστρέφει τη ροή του προγράμματος στον έλεγχο της συνθήκης σε ένα βρόγχο.
  - Βολικό για τον έλεγχο συνθηκών πριν ξεκινήσει η εκτέλεση του βρόγχου, ή για πρόωρη επιστροφή στον έλεγχο της συνθήκης

- break: Μας βγάζει έξω από την εκτέλεση του βρόχου από οποιοδήποτε σημείο μέσα στον κώδικα.
  - Βολικό για να σταματάμε το βρόγχο όταν κάτι δεν πάει καλά.

- Κάποιοι θεωρούν οι εντολές αυτές χαλάνε το μοντέλο του δομημένου προγραμματισμού.

Οι εντολές break και continue

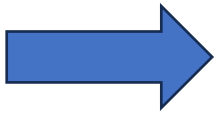


Παράδειγμα

```
while (…)
{
   if (everything is ok){

     < rest of code>

   }// end of if
} // end of while loop
```



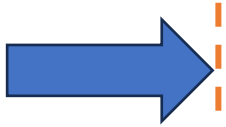
```
while (…)
{
   if (I don't like something){
        continue;
   }

   < rest of code>

} // end of while loop
```

```
while (… && !StopFlag)
{
   < some code >

   if (I should stop){
     StopFlag = true;
   }else{
     < some more code>
   }
} // end of while loop
```



```
while (…)
{
   < some code>

   if (I should stop){
        break;
   }

   < some code>
} // end of while loop
```

```java
import java.util.Scanner;

class FlowTestContinue
{
    public static void main(String[] args)
    {
        Scanner reader = new Scanner(System.in);
        int inputInt = reader.nextInt();
        while (inputInt != 0)
        {
            if (inputInt%2 == 0){
                inputInt = reader.nextInt();
                continue;
            }
            if (inputInt < 0 ){
                for (int i = inputInt; i < 0; i++)
                {
                    System.out.println("Counter = " + i);
                }
            } else if (inputInt > 0){
                for (int i = inputInt; i > 0; i--)
                {
                    System.out.println("Counter = " + i);
                }
            }
            inputInt = reader.nextInt();
        }
    }
}
```

Η αντίστροφη μέτρηση εκτελείται μόνο για περιττούς αριθμούς

Εμβέλεια (scope) μεταβλητών

- Προσέξτε ότι η μεταβλητή `int i` πρέπει να οριστεί σε κάθε for, ενώ η `inputInt` πρέπει να οριστεί έξω από το while-loop αλλιώς ο compiler διαμαρτύρεται
  - Προσπαθούμε να χρησιμοποιήσουμε μια μεταβλητή εκτός της εμβέλειας της
- Η κάθε μεταβλητή που ορίζουμε έχει εμβέλεια (scope) μέσα στο block το οποίο ορίζεται
  - Τοπική μεταβλητή μέσα στο block
- Μόλις βγούμε από το block η μεταβλητή χάνεται
  - Ο compiler δημιουργεί ένα χώρο στη μνήμη για το block το οποίο εκτελούμε, ο οποίος εξαφανίζεται όταν το block τελειώσει
- Ένα block μπορεί να περιλαμβάνει κι άλλα φωλιασμένα blocks
  - Η μεταβλητή έχει εμβέλεια και μέσα στα φωλιασμένα blocks
  - Δεν μπορούμε να ορίσουμε άλλη μεταβλητή με το ίδιο όνομα σε ένα φωλιασμένο block

Παράδειγμα με το scope μεταβλητών

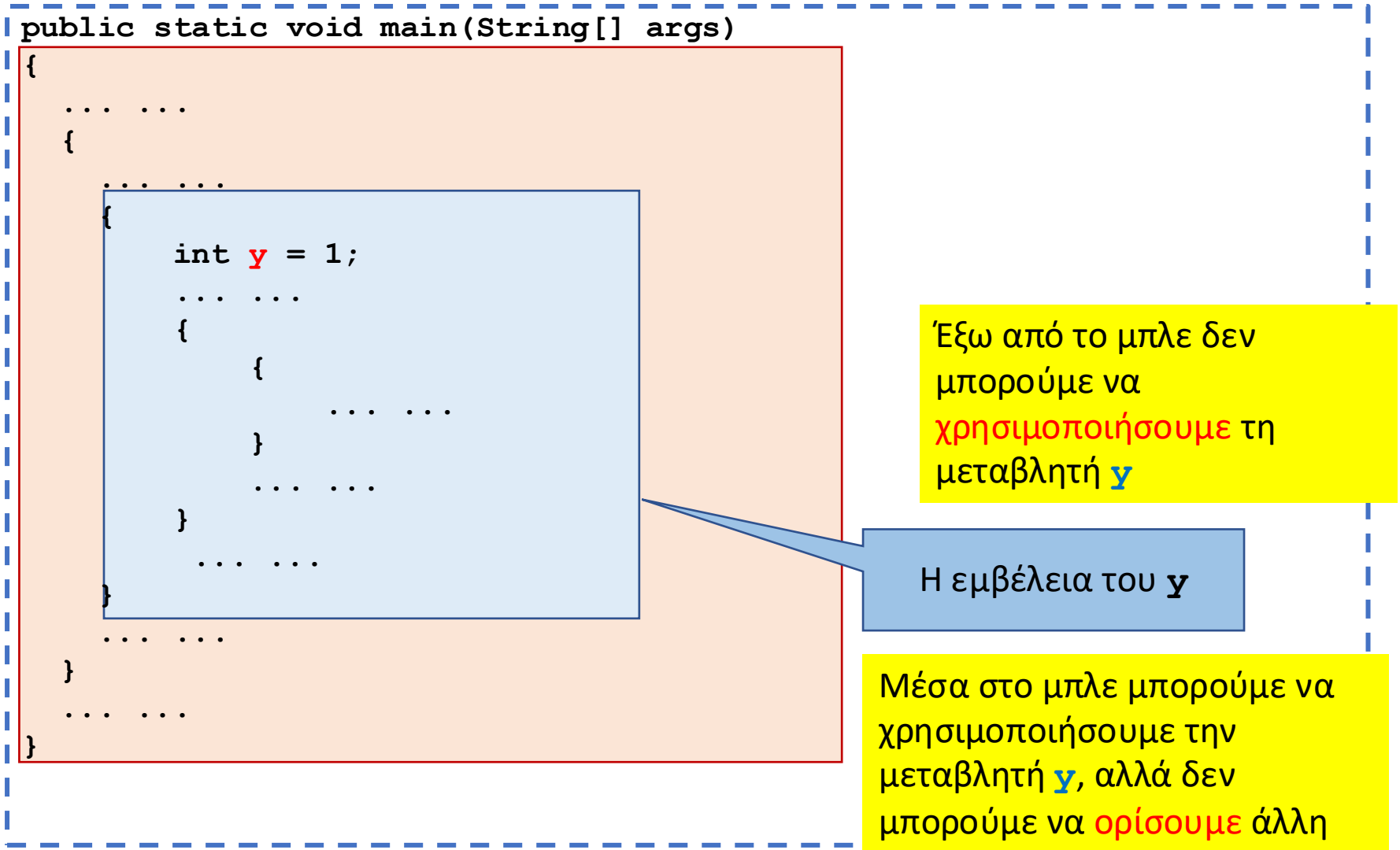
```
1   public static void main(String[] args)
2   {
3         int y = 1;
4         int x = 2;
5         for (int i = 0; i < 3; i++)
6         {
7            y = i;
8            double x = i+1;
9            int z = x+y;
10           System.out.println("i = " + i);
11           System.out.println("y = " + y);
12           System.out.println("z = " + z);
13        }
14        System.out.println("i = " + i);
15        System.out.println("z = " + z);
16        System.out.println("y = " + y);
17        System.out.println("x = " + x);
18  }
```

Ο κώδικας έχει λάθη σε τρία σημεία

Παράδειγμα με το scope μεταβλητών

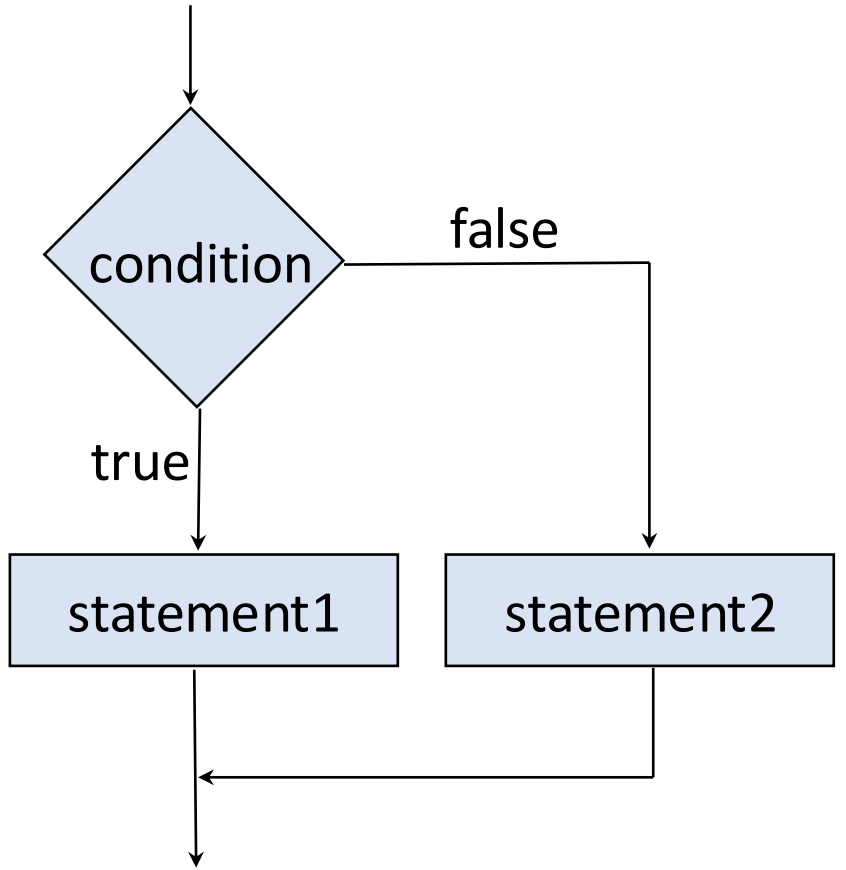
```
1   public static void main(String[] args)
2   {
3         int y = 1;
4         int x = 2;
5         for (int i = 0; i < 3; i++)
6         {
7            y = i;
8            double x = i+1;
9            int z = x+y;
10           System.out.println("i = " + i);
11           System.out.println("y = " + y);
12           System.out.println("z = " + z);
13        }
14        System.out.println("i = " + i);
15        System.out.println("z = " + z);
16        System.out.println("y = " + y);
17        System.out.println("x = " + x);
18  }
```

Ο κώδικας έχει λάθη σε τρία σημεία


public static void main(String[] args)
{
... ...
{
... ...
{
int y = 1;
... ...
{
{
... ...
}
... ...
}
... ...
}
... ...
}
... ...
}
Η εμβέλεια του y
Έξω από το μπλε δεν μπορούμε να χρησιμοποιήσουμε τη μεταβλητή y
Μέσα στο μπλε μπορούμε να χρησιμοποιήσουμε την μεταβλητή y, αλλά δεν μπορούμε να ορίσουμε άλλη μεταβλητή με το όνομα y

Το if-else statement

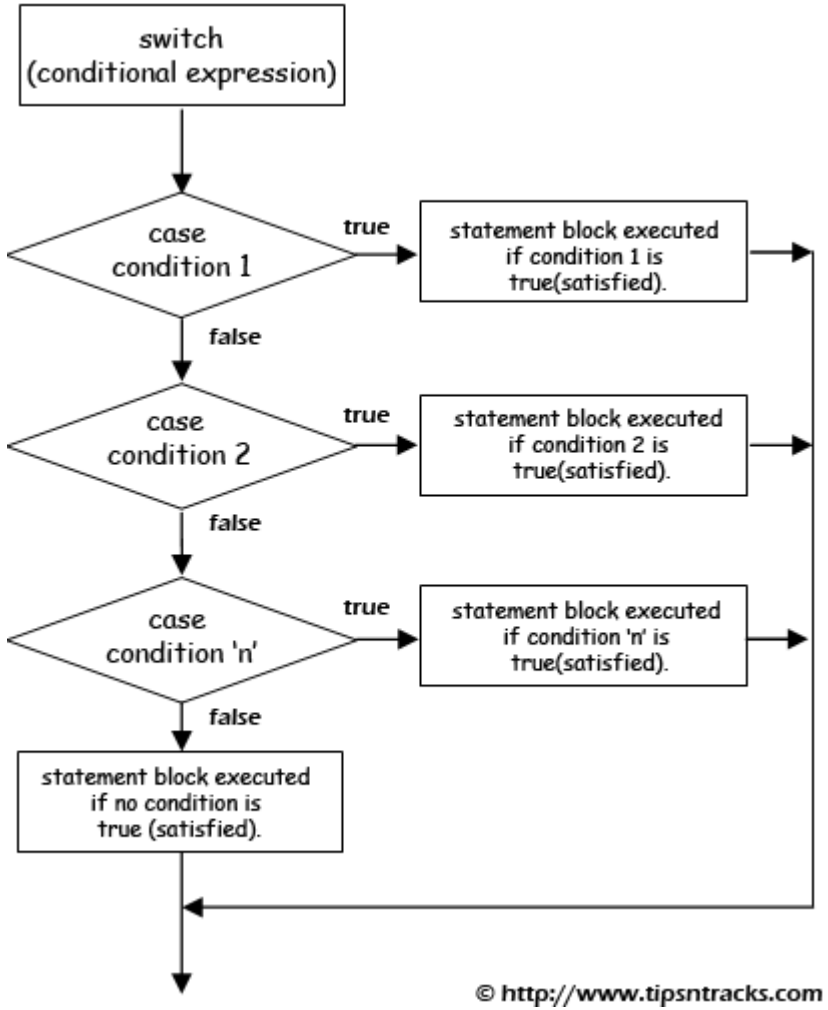
- Το if-else statement δουλεύει καλά όταν στο condition θέλουμε να περιγράψουμε μια επιλογή με δύο πιθανά ενδεχόμενα.
- Τι γίνεται αν η συνθήκη μας έχει πολλά ενδεχόμενα?



Switch statement

Συντακτικό:

```
switch (<condition expression>){
  case <condition 1>:
    code statements 1
    break;
  case <condition 2>:
    code statements 2
    break;
  case <condition 3>:
    code statements 3
    break;
  default:
    default statements
    break;
}
```



Παράδειγμα

- Ένα πρόγραμμα που να εύχεται καλημέρα σε τρεις διαφορετικές γλώσσες ανάλογα με την επιλογή του χρήστη

```
import java.util.Scanner;

class SwitchTest{
  public static void main(String args[])
  {
    Scanner input = new Scanner(System.in);
    String option = input.next();

    switch(option){
      case "GR":
          System.out.println("kalimera");
          break;
      case "EN":
          System.out.println("good morning");
          break;
      case "FR":
          System.out.println("bonjour");
          break;
      default:
          System.out.println("I do not speak this language.\n" +
                      "Greek, English, French only");
    }
  }
}
```

ΠΙΝΑΚΕΣ

Πίνακες

- Πολλές φορές έχουμε πολλές μεταβλητές του ίδιου τύπου που συσχετίζονται και θέλουμε να τις βάλουμε μαζί
  - Τα ονόματα των φοιτητών σε μία τάξη
  - Οι βαθμοί ενός φοιτητή για όλα τα εργαστήρια
- Για το σκοπό αυτό χρησιμοποιούμε τους πίνακες

Πίνακες

- Ορισμός πίνακα:

```
int[] myArray1  = {10,20};
int myArray2[] = new int[2];
int[] myArray3;
```

Ορίζει ένα πίνακα ακεραίων δύο θέσεων με αρχικές τιμές 10,20

Ορίζει ένα πίνακα ακεραίων δύο θέσεων χωρίς αρχικές τιμές (αρχικοποιούνται αυτόματα στο μηδέν)

Ορίζει μια μεταβλητή που είναι πίνακας από ακεραίους αλλά δεν δεσμεύει χώρο για τον πίνακα

- Οι πίνακες ορίζονται με ένα μέγεθος (length) και αυτό δεν αλλάζει εκτός αν ορίσουμε ξανά τον πίνακα

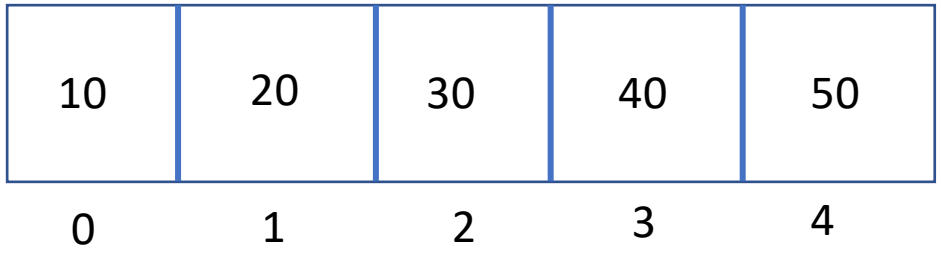
- Στη Java ένας πίνακας είναι ένα αντικείμενο και έχει properties
  - `System.out.println(myArray2.length);`
  - Τυπώνει το μέγεθος του πίνακα

Πρόσβαση Των Στοιχείων Του Πίνακα

- Προσοχή!

  Τα στοιχεία του πίνακα αριθμούνται από το `0…length-1` (ΟΧΙ `1…length`)

  - `int myArray[] = {10,20,30,40,50};`



- Για να προσπελαύνουμε το δεύτερο στοιχείο του πίνακα
  - `myArray[1] += 5;`
  - `System.out.println(myArray[1]);`

Διατρέχοντας Ένα Πίνακα

- Στην Java έχουμε δύο τρόπους να διατρέχουμε ένα πίνακα

Διατρέχουμε τα στοιχεία

```
for (<array type> element: array)
{
  … do something with element…
}
```

```
int array[] = {1,3,5,7};
for (int element: array)
{
  System.out.println(element)
}
```

Διατρέχουμε τις θέσεις του πίνακα

```
for (int i = 0; i < array.length; i++)
{
  … do something with array[i]…
}
```

```
int array[] = {1,3,5,7};
for (int i = 0; i < array.length; i++)
{
  System.out.println(array[i])
}
```

Πίνακες

```
public class TestArrays1 {
    public static void main(String[] args){

        int arr0[];  // int[] arr0;

        int arr1[] = {1, 2, 3, 4};
        for (int i = 0; i < arr1.length; i++){
            System.out.println(arr1[i]);
        }

        int arr2[] = new int[10];
        for (int i = 0; i < arr2.length; i++){
            arr2[i] = i+1;
        }
        arr0 = arr2;

    }
}
```

Εναλλακτικό συντακτικό

Παράδειγμα

- Τυπώστε όλα τα στοιχεία του πίνακα και όλα τα ζεύγη από στοιχεία στον πίνακα

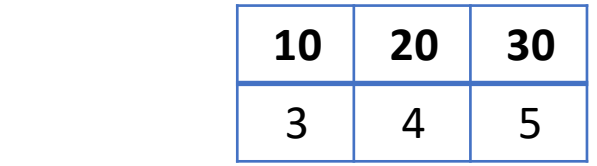
```
class ScanArray
{
    public static void main(String[] args)
    {
        double[] array = {5.3, 3.4, 2.3, 1.2, 0.1};

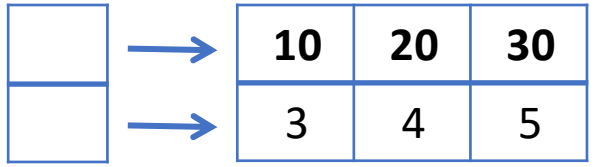
        // Print all elements
        for (double element: array){
            System.out.println(element);
        }

        // Print all pairs of elements
        for (int i = 0; i < array.length; i++){
            for (int j = i+1; j < array.length; j++){
                System.out.println(array[i] + " " + array[j]);
            }
        }
    }
}
```

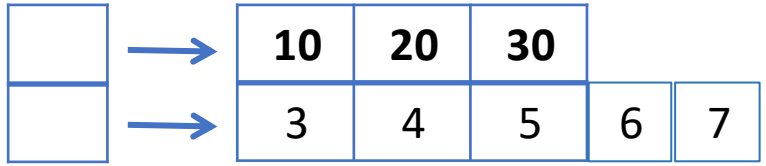
Πολυδιάστατοι Πίνακες

- Μπορούμε να ορίσουμε και πολυδιάστατους πίνακες
  - `int[][] myArray1 = {{10,20,30},{3,4,5}};`
  - `int[][] myArray2 = new int[2][3];`
- Ένας δισδιάστατος πίνακας είναι ένας πίνακας από αντικείμενα-πίνακες
  - `int[][] myArray3 = new int[2][]`
  - `myArray3[0] = new int[3]`
  - `myArray3[1] = new int[3]`
- Ο πίνακας μπορεί να είναι ασύμμετρος
  - `myArray3[1] = new int[5]`
- Τι παίρνω για τα παρακάτω?
  - `System.out.println(myArray3.length);`
  - `System.out.println(myArray3[1].length);`





Η κάθε γραμμή είναι ένας πίνακας και πρέπει να αρχικοποιηθεί



Πίνακες

```
public class TestArrays2 {
        public static void main(String[] args){
                int[][] arr3 = {{1, 2, 3}, {3, 4, 5}};

                int[][] arr4 = new int[10][20];

                arr4 = arr3;

                System.out.println(arr4.length + " " + arr4[0].length);

                int arr5[][] = new int[2][];
                arr5[0] = new int[3];
                arr5[1] = new int[5];
        }
}
```

Πίνακας με αρχικές τιμές

Πίνακας 10x20

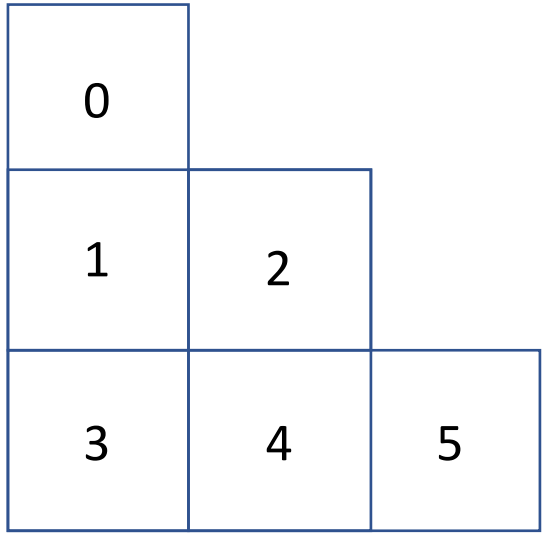
Ο πίνακας arr4 γίνεται ίδιος με τον arr3
Δηλαδή δείχνει στον ίδιο χώρο μνήμης

Τυπώνει “2 3”

Ασύμμετρος πίνακας

Αρχικοποίηση Πινάκων

- Δημιουργήστε τους παρακάτω πίνακες:
  - Ένα μονοδιάστατο πίνακα με n θέσεις με τις τιμές 0..n-1
  - Ένα δισδιάστατο πίνακα με nxn θέσεις με τις τιμές 0…n^2-1
  - Ένα κάτω διαγώνιο πίνακα nxn (π.χ. παρακάτω 3x3)
- Το μέγεθος του πίνακα θα το δίνει ο χρήστης



```
import java.util.Scanner;

class ArrayInitialization
{
	public static void main(String[] args){
		Scanner input = new Scanner(System.in);
		int n = input.nextInt();

		int[] array1d = new int[n];
		for (int i = 0; i < n; i++){
			array1d[i] = i;
		}
		for (int i = 0; i < n; i++){
			System.out.print(array1d[i] + " ");
		}
		System.out.println();
	}
}
```

```
import java.util.Scanner;

class ArrayInitialization
{
	public static void main(String[] args){
		Scanner input = new Scanner(System.in);
		int n = input.nextInt();

		int[][] array2d = new int[n][n];
		for (int i = 0; i < n; i++){
			for (int j = 0; j < n; j++){
				array2d[i][j] = i*n+j;
			}
		}
		for (int i = 0; i < n; i++){
			for (int j = 0; j < n; j++){
				System.out.print(array2d[i][j] + " " );
			}
			System.out.println();
		}
	}
}
```

```
import java.util.Scanner;

class ArrayInitialization
{
        public static void main(String[] args){
                Scanner input = new Scanner(System.in);
                int n = input.nextInt();

                int[][] lowerDiagonal = new int[n][];
                for (int i = 0; i < n; i++){
                    lowerDiagonal[i] = new int[i+1];
                    for (int j = 0; j < i+1; j++){
                       lowerDiagonal[i][j] = i*(i+1)/2 + j;
                    }
                }
                for (int i = 0; i < n; i++){
                    for (int j = 0; j < i+1; j++){
                       System.out.print(lowerDiagonal[i][j] + " ");
                       }
                    System.out.println();
                }
        }
}
```